FinSecure: Trust-Aware Zero-Trust Framework

Continuous Behavioral Security Architecture for Autonomous Multi-Agent Banking Systems

TECHNICAL SPECIFICATION & SECURITY BLOCK VERIFICATION REPORT

1. Executive Summary & Design Philosophy

Autonomous Multi-Agent Systems (MAS) powered by Large Language Models (LLMs) represent the frontier of complex enterprise automation. However, in mission-critical environments such as banking operations, multi-agent architectures introduce existential vulnerabilities. Standard perimeter security (e.g., authenticating a user at login) fails because **once inside, agents trust each other implicitly**. An attacker can plant an indirect prompt injection in an invoice memo, customer KYC document, or compliance policy. When an agent reads that data, it becomes a *Confused Deputy*, initiating unauthorized fund transfers or escalating privileges across peer agents.

FinSecure solves this via the principle of **Complete Mediation**: Agents never talk directly to each other, never possess direct database connections, and cannot call external APIs independently. Every single inter-agent message and every tool invocation is packaged into a cryptographically signed envelope and validated at runtime by a centralized **Zero-Trust Security Gateway**.

> **The Complete Mediation Principle**
>
> Zero-Trust requires that every transaction is verified continuously at every step, not just at session initiation. Every envelope must prove its sender's cryptographic identity, prove that its action was authorized by the human user's original request, satisfy declarative role-based and attribute-based security policies, pass burst limits, and comply with non-linear behavioral trust score requirements.

2. High-Level Website Walkthrough (The Demonstration Surface)

When accessing the FinSecure platform at `http://127.0.0.1:8000`, the web dashboard serves as an interactive Security Operations Center (SOC) specifically tailored to demonstrate multi-agent behavioral security:

Dashboard View	What the User Sees	Underlying Security Functionality
1. Overview	High-level bank metrics (total active accounts, balance volume, fraud cases), gateway decision counts (Approved, Blocked, Escalated, Isolated), and live trust gauges per agent.	Real-time telemetry aggregated from SQLite and the in-memory trust engine, showing current posture across the agent mesh.
2. Operations	Staff prompt input box (e.g. "*Transfer Rs 25,000 from ACC-1002 to ACC-1012*"). Below it renders the **Checkpoint Strip** for every message hop.	Visualizes envelope evaluation: Identity, Chain, Scope, OPA Policy, Prompt Shield, Burst Limiter, Trust. Shows how each agent hop is vetted.
3. Bank Data	Interactive tables for Customers, Accounts, Transactions, Loans, Fraud Cases, and Knowledge Base Policies.	Displays raw bank records before and after attacks. Staff can inspect injected memos or poisoned KYC documents.
4. Pending Reviews	Human-in-the-Loop review queue displaying irreversible actions held for human approval (Approve / Reject buttons).	When an agent attempts a Tier-2 irreversible action (e.g. `unfreeze_account` or high-value transfer) with borderline trust or tainted lineage, the Decision Engine issues an `ESCALATE` verdict.
5. Security Console	Live, streaming terminal-style feed of decisions, alert banners, blocked envelopes, and isolation events.	Powered by an asynchronous WebSocket (`/ws`) pushing events instantly from the gateway's event bus.
6. Delegation Graph	Visual Directed Acyclic Graph (DAG) of the current run, color-coded by verdict (green = approved, red = blocked, orange = escalated).	Renders the cryptographic Provenance DAG showing how the root Intent Token delegated execution through the Planner, Researcher, and Executor.
7. Trust Monitor	Interactive charts tracking each agent's trust score over time against isolation and restriction thresholds.	Visualizes the non-linear trust formula vs. traditional linear moving average, proving why linear trust fails against reputation laundering.
8. Audit Trail	Chronological cryptographic Merkle ledger. Features a **"Tamper with Row"** button and a **"Verify Ledger"** button.	Allows staff to simulate insider database tampering. Verifying re-hashes rows against signed roots stored outside SQLite, pinpointing modified rows.
9. Attack Lab	Catalog of 6 built-in attack scenarios (Prompt Injection, Confused Deputy, Reputation Laundering, Burst Flood, Audit Tampering, Rogue Identity).	Runs attacks side-by-side in Baseline vs. Secured mode to visually demonstrate attacks succeeding in standard systems and failing in FinSecure.
10. Settings & Mode Toggle	Master **Secured / Baseline** switch, LLM provider selection (Groq / Ollama / Mock), and individual module checkboxes.	Facilitates ablation studies (e.g. turning off the Prompt Shield or Burst Limiter to demonstrate individual module importance).

3. The Dual-Track Security Proxy Layer

The Zero-Trust Security Proxy is divided into two distinct execution tracks to resolve the conflict between complete mediation and performance:

3.1 Synchronous Critical Path (Blocking Inline Enforcement)

Implemented in `backend/gateway/gateway.py` in the `_process()` method:

1. **Identity & Certificate Check:** Validates the sender's X.509 certificate against the internal CA, checks the revocation list, and verifies the RSA-PSS signature.
2. **Provenance DAG Verification:** Walks parent node hashes back to the root Intent Token to verify lineage authenticity.
3. **Intent Scope Authorization:** Checks whether the requested tool, entities (accounts/loans), and monetary amounts match the human request's scope.
4. **OPA Policy Engine:** Evaluates declarative Rego RBAC (tool permissions) and ABAC rules (account active status, transfer limits, loan credit scores).
5. **Prompt Shield:** Scans tool inputs and messages using regex heuristics and Llama Prompt Guard 2 (86M). Redacts high-risk injection payloads.
6. **Burst Limiter:** Verifies sliding-window request frequency and transfer volume.
7. **Adaptive Decision Engine:** Synthesizes all checks into a definitive verdict (`APPROVE`, `RESTRICT`, `ESCALATE`, `BLOCK`, `ISOLATE`).

Latency Guarantee: The critical path only performs fast cryptographic checks, cached policy lookups, and lightweight classifications (typically < 8ms). The request thread is blocked until the decision is rendered; unapproved actions are rejected before execution.

3.2 Asynchronous Analytics Track (Non-Blocking Intelligence)

Implemented in `backend/gateway/gateway.py` lines 58–116 using a dedicated thread-safe queue and daemon worker thread:

- **Trust Score Updates:** Successful actions enqueue a reward computation. The mathematical trust formula runs asynchronously without slowing down agent responses.
- **Behavioral Anomaly Detection:** Statistical profiling of transaction amounts runs in the background. Outliers trigger background trust penalties.
- **Merkle Ledger Sealing:** Every 50 events, pending audit leaves are aggregated into a Merkle tree; the root is signed and committed to disk asynchronously.
- **Synchronization Barrier:** The `gateway().flush()` method blocks until the queue drains, ensuring deterministic consistency during automated evaluations and testing.

4. Mathematical Formulations & Parameter Definitions

The Behavioral Trust Engine (`backend/security/behaviour.py`) provides mathematical defense against **Reputation Laundering**—an attack where an agent compromised by an adversary intentionally executes dozens of trivial benign operations (e.g., reading account balances) to artificially inflate its trust score before launching an attack.

Formula 1: Multiplicative Penalty Decay on Violation

$$T_{new} = T \cdot (1 - s)$$

$$T_v = T_{new}, \quad t_v = now$$

Definitions & Meanings:

• **T**: Current trust score of the agent, bounded in [0.0, 1.0]. Initialized at 0.75.

• **s**: Severity weight of the violation: *low* = 0.20 (burst limits), *medium* = 0.40 (policy deny), *high* = 0.60 (intent scope breach), *identity* = 1.00 (signature forgery).

• **T_V**: Anchor trust level immediately after the violation.

• **t_v**: Unix epoch timestamp when the violation occurred.

Impact: Asymmetric punishment. A single high-severity violation drops trust from 0.75 to 0.30 immediately, blocking irreversible actions.

Formula 2: Anti-Gaming Dampened Recovery Reward

$$T_{new} = \min(T + \eta \cdot w_a \cdot (1 - T), \quad C(t))$$

where: $\eta = \eta_0 / (1 + n_W(t))$

Definitions & Meanings:

• **η_0**: Base learning rate / trust reward sensitivity (configured at 1.0).

• **$n_W(t)$**: The number of successful good actions executed by the agent within sliding time window **W** (configured at 300 seconds / 5 mins).

• **η**: **Anti-Gaming Coefficient**. As an agent spams more actions within window W, n_W increases, which causes η to rapidly decay towards zero. Packing 30 actions together yields virtually zero reward.

• **w_a**: Action weight based on effort and criticality: *read* = 0.01, *tier-1 write* = 0.03, *tier-2 write* = 0.05, *message* = 0.005. Cheap read queries earn almost no trust compared to high-risk verified actions.

• **(1 - T)**: Diminishing returns factor. As trust approaches 1.0, marginal gains asymptotically decrease.

Formula 3: Non-Linear Time-Bounded Recovery Ceiling

$$C(t) = T_v + (1 - T_v) \cdot (1 - e^{-(t - t_v) / \tau})$$

Definitions & Meanings:

• **C(t)**: Hard upper bound (ceiling) on the agent's maximum attainable trust score at elapsed time *t*.

• **$t - t_v$**: Real elapsed wall-clock seconds since the last security violation occurred.

• **τ (tau)**: Time constant governing recovery velocity (configured at 1,800 seconds / 30 minutes).

Impact: **Trust cannot be laundered with volume; it can only be redeemed with time.** Even if an adversary generates 100 successful operations in 60 seconds, C(t) remains pinned near T_V, preventing the trust score from climbing above the threshold required to execute high-impact fund transfers.

Formula 4: Provenance DAG Ancestry Commitment

$$\text{NodeHash} = \text{SHA-256}(\text{PayloadHash} \,||\, \text{Sort}(\text{ParentNodeHashes}))$$

Definitions & Meanings:
• **PayloadHash**: SHA-256 digest of the canonical JSON payload (tool name, arguments, or message content).
• **ParentNodeHashes**: Concatenated, sorted hashes of all causal parent envelopes in the delegation graph.
Impact: Creates an immutable cryptographic merkle-dag. An adversary cannot modify a middle node in the agent chain without invalidating all downstream node hashes, guaranteeing complete tamper-detection across agent hops.

Formula 5: Merkle Tree Audit Ledger Inclusion Proof

$$\text{Leaf}_i = \text{SHA-256}(\text{CanonicalJSON}(\text{seq}, \text{timestamp}, \text{event_type}, \text{actor}, \text{data}))$$

$$\text{Root} = \text{MerkleRoot}([\text{Leaf}_0, \text{Leaf}_1, \ldots, \text{Leaf}_{N-1}])$$

Definitions & Meanings:
• Every audit event is serialized deterministically (sorted keys, compact delimiters) and hashed into a leaf.
• Leaves are paired and hashed recursively: $H_{parent} = \text{SHA-256}(H_{left} \,||\, H_{right})$.
• Merkle roots are signed by the Ledger's RSA key and committed to an external append-only file (`data/ledger_roots.jsonl`) outside the database. Modifying, inserting, or deleting a single row in SQLite causes tree re-computation to diverge from the signed root.

5. Adaptive Decision Engine: Priority Rules & Action Authorizations

Implemented in `backend/security/decision.py`. When the synchronous pipeline finishes checking an envelope, it evaluates nine deterministic rules top-to-bottom. The first rule that matches determines the verdict:

Rule	Condition	Verdict	Operational Outcome
1	Identity validation fails, bad signature, or certificate revoked	**BLOCK**	Envelope discarded. Security alert emitted. Sender role cannot be authenticated.
2	Delegation chain broken or tool call outside signed Intent scope	**BLOCK**	Tool invocation aborted. Prevents Confused Deputy attacks (e.g., executing transfer during inquiry).
3	OPA policy denies (RBAC or ABAC violation)	**BLOCK**	Action rejected. Enforces limits, prevents inactive account transfers, prevents PII leakage.
4	Prompt Shield detects high-risk injection payload	**BLOCK**	Malicious instructions redacted or envelope blocked; intent chain marked tainted.
5	Agent trust < 0.20 or ≥ 3 repeated blocks within 10 minutes	**ISOLATE**	Agent X.509 certificate revoked. Agent is quarantined and cannot send messages or call tools.
6	Burst limiter tripped (exceeded frequency or velocity quota)	**RESTRICT**	Agent placed into 120-second cooldown. Non-critical requests rejected.
7	Tier-2 (irreversible) tool call while trust score < 0.40	**BLOCK**	Irreversible writes prohibited for untrusted or penalized agents.
8	Tier-2 tool call with trust < 0.70, tainted chain, or policy obligation	**ESCALATE**	Action suspended and placed into Pending Reviews table for human supervisor authorization.
9	All security checks passed	**APPROVE**	Envelope delivered to target agent or Python banking tool executed against database.

6. Attack Surface Coverage & Verification

Threat Class	Real-World Failure Mode	FinSecure Defense Guarantee
Confused Deputy	Harmless user query (e.g. policy lookup) reads poisoned text instructing the agent to transfer Rs 50,000.	**Intent Token Root Scope:** Initial prompt sets allowed writes to `[]`. Tool call to `transfer_funds` violates intent scope and triggers Rule 2 `BLOCK`.
Reputation Laundering	Attacker executes 40 benign queries after a violation to reset moving-average trust.	**Anti-Gaming η & Recovery Ceiling C(t):** Recovery is throttled by real elapsed time (τ = 30 min) and action frequency. Spammed actions yield zero trust gain.
Prompt Injection	Attacker embeds adversarial instructions in transaction memos or KYC documents.	**Dual-Stage Prompt Shield:** Llama Prompt Guard 2 detects attack tokens; spotlighting wraps data in markers prohibiting execution.
Audit Tampering	Insider edits database records to alter transaction amounts or hide illicit activity.	**Signed Merkle Ledger:** Recomputing proofs against external signed roots detects tampering and identifies the exact edited row.

Conclusion

FinSecure establishes continuous behavioral security across autonomous LLM multi-agent operations. By combining cryptographic identities, DAG provenance lineage, declarative policy evaluation, non-linear trust dynamics, and tamper-evident Merkle auditing, FinSecure ensures that multi-agent systems remain verifiably secure against modern autonomous threats.